TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 3
Diseño de interfaz y navegación

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 3

Qué vamos a ver

Objetivos

- Construir pantallas con los componentes nativos.
- Dar estilo y tema a la aplicación.
- Adaptarla a distintos tamaños y orientaciones.
- Que se pueda usar sin mirar la pantalla.
- Navegar entre pantallas y proteger rutas.

Partes

- Construcción de pantallas
- Estilos, temas y diseño responsivo
- Gestos y animaciones
- Accesibilidad
- Navegación (deck aparte)

UNIDAD 3 · PARTE 1

Construcción de pantallas

Los componentes con los que se arma todo lo demás.

Los componentes que vas a usar siempre

`View`
El contenedor. Equivale al `<div>`, pero ya viene en Flexbox con dirección `column`.

`Text`
Todo texto visible va adentro. No se puede poner texto suelto en una `View`.

`TextInput`
Entrada de datos. Sin `form`: el estado lo manejás vos.

`Pressable`
Cualquier cosa que responda al toque. Reemplaza a los viejos `Touchable*`.

> El error más común de quien viene de web: **texto fuera de un `<Text>`**. React Native lanza un error en tiempo de ejecución, no lo ignora como el navegador.

`View` · el contenedor

Mapea directo al contenedor nativo — `UIView` en iOS, `android.view` en Android — y admite de cero a muchos hijos de cualquier tipo. Ya viene en Flexbox con dirección `column`; el detalle de estilos lo vemos en la Parte 2. Lo propio de `View` son estas cuatro props.

`onLayout`	avisa la posición y el tamaño reales apenas se calculan: es la forma de medir algo en lugar de suponerlo.
`hitSlop`	agranda el área sensible al toque sin agrandar la vista. Nunca se pasa de los límites del padre.
`pointerEvents`	`none` la vuelve intocable, `box-none` deja pasar el toque a los hijos, `box-only` al revés.
`role` y `accessibilityLabel`	qué es y cómo se llama para el lector de pantalla. Vuelve en la Parte 4.

> React Native **borra del árbol nativo las `View` que sólo aportan layout**. Por eso ponerle `testID` o `nativeID` desactiva esa optimización — y por eso a veces una vista no aparece en el inspector nativo.

`Text` · mostrar texto

Es el único que acepta strings. Usa **layout de texto**, no Flexbox: no se comporta como una caja, se corta al final de la línea. Y anidar `Text` adentro de `Text` es la forma de mezclar formatos en un mismo párrafo — por debajo se aplana en un solo texto con atributos.

```
<Text style={e.parrafo}>
  <Text style={{ fontWeight: 'bold' }}>Total: </Text>
  $1.250 con<Text style={{ color: 'crimson' }}> 20% off</Text>
</Text>

<Text numberOfLines={2} ellipsizeMode="tail">{descripcion}</Text>
<Text onPress={abrirTerminos}>Ver términos</Text>
```

> `allowFontScaling` viene en `true`: si la persona agrandó la letra del sistema, tu texto crece. **No lo apagues** — si te rompe el diseño, acotalo con `maxFontSizeMultiplier`.

`TextInput` · entrada de datos

Es un **componente controlado**: el valor vive en tu estado y se lo devolvés por `value`. `onChangeText` te entrega el texto ya listo, sin evento de por medio.

```
<TextInput value={texto} onChangeText={setTexto}
           placeholder="Escribí algo" />
```

`keyboardType`	qué teclado abre: `email-address`, `numeric`, `phone-pad`…
`autoCapitalize`	viene en `sentences`: para mails y usuarios va `none`.
`autoComplete`	ya es multiplataforma; es lo que hace que el gestor de contraseñas ofrezca completar.
`secureTextEntry`	oculta lo escrito. No se combina con `multiline`.
`returnKeyType` + `onSubmitEditing`	qué dice la tecla de retorno y qué pasa al tocarla — así se encadenan campos.

> `blurOnSubmit` quedó **deprecado**: ahora es `submitBehavior`, con `'submit'`, `'blurAndSubmit'` o `'newline'`. Buena parte de los tutoriales todavía usa el viejo.

`Pressable` · responder al toque

Envuelve cualquier cosa y detecta las distintas etapas del toque. Reemplaza a los viejos `Touchable*`, que ya no conviene usar.

```
<Pressable
  onPress={agregar}
  onLongPress={mostrarOpciones}
  hitSlop={10}
  android_ripple={{ color: '#DDD' }}
  style={({ pressed }) => [e.boton, pressed && e.botonTocado]}
>
  {({ pressed }) => (
    <Text>{pressed ? 'Agregando…' : 'Agregar'}</Text>
  )}
</Pressable>
```

> `style` y `children` pueden ser **funciones** que reciben `{ pressed }`: ahí se dibuja el estado de presionado, sin manejar estado propio.

Edge-to-edge OBLIGATORIO DESDE SDK 54

Desde Android 15 y Expo SDK 54, **tu app dibuja debajo de la barra de estado y de la barra de navegación**. No se puede desactivar.



Sin manejarlo: el reloj se superpone al título



Con `insets`: el contenido arranca donde debe

> `react-native-safe-area-context` ya viene instalado si usás Expo Router. **El error típico es usarlo sólo en la pantalla raíz:** hay que aplicarlo en cada pantalla que toque un borde.

Safe area en la práctica

Dos formas, misma librería: `react-native-safe-area-context`.

`SAFEAREAVIEW` · EL COMPONENTE

```
import { SafeAreaView }
  from 'react-native-safe-area-context';

<SafeAreaView
  style={{ flex: 1 }}
  edges={['top', 'bottom']}
>
  <Text>Contenido</Text>
</SafeAreaView>
```

Envuelve y aplica el margen solo. Con `edges` elegís qué lados. Es lo que vas a usar casi siempre.

`USESAFEAREAINSETS` · EL HOOK

```
import { useSafeAreaInsets }
  from 'react-native-safe-area-context';

const insets = useSafeAreaInsets();

<FlatList
  contentContainerStyle={{
    paddingBottom: insets.bottom,
  }}
/>
```

Te da los números y decidís dónde aplicarlos. Necesario cuando el margen no va en el contenedor.

Cuidado con el import: también existe un `SafeAreaView` en `react-native`, pero sólo funciona en iOS y está en desuso. **Siempre el de `react-native-safe-area-context`.**

Desplazar contenido: `ScrollView`

Una `View` no se desplaza: lo que no entra en pantalla queda cortado. `ScrollView` es el contenedor que sí desplaza, y sirve para **cualquier contenido** — un formulario largo, una pantalla de detalle, un carrusel con `horizontal`.

```
<ScrollView
  style={{ flex: 1 }}
  contentContainerStyle={{ padding: 16, gap: 12 }}
>
  <Image source={{ uri: producto.foto }} style={estilos.foto} />
  <Text style={estilos.titulo}>{producto.nombre}</Text>
  <Text>{producto.descripcion}</Text>
  <Pressable onPress={agregar}><Text>Agregar</Text></Pressable>
</ScrollView>
```

> Renderiza **todos** sus hijos de una, se vean o no. Va bien con contenido variado y de cantidad conocida; para una **colección que puede crecer** no alcanza, y ahí entran las listas de las próximas dos slides.

Listas · `FlatList`

Para una **colección de elementos** hay dos componentes, `FlatList` y `SectionList`. Los dos **renderizan sólo lo que se ve** (y un poco más): montan y desmontan las filas a medida que desplazás. `FlatList` es el que vas a usar casi siempre.

```
<FlatList
  data={productos}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <Tarjeta producto={item} />}
  ListEmptyComponent={<Vacio />}
  onEndReached={cargarMas}
  refreshControl={
    <RefreshControl refreshing={cargando} onRefresh={recargar} />
  }
/>
```

> Poner una lista larga adentro de un `ScrollView` es el error de rendimiento más frecuente: se renderizan los mil elementos aunque se vean diez. Tampoco anides una `FlatList` dentro de un `ScrollView` vertical: pierde la virtualización y React Native te lo avisa por consola.

Listas · `SectionList`

La misma virtualización que `FlatList`, pero los datos vienen partidos en grupos y cada grupo lleva su encabezado.

```
const secciones = [
  { title: 'Bebidas', data: [agua, jugo] },
  { title: 'Snacks',  data: [mani, papas] },
];

<SectionList
  sections={secciones}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <Tarjeta producto={item} />}
  renderSectionHeader={({ section }) => (
    <Text style={estilos.encabezado}>{section.title}</Text>
  )}
  stickySectionHeadersEnabled
/>
```

Cuál uso: contenido variado que no entra en pantalla → `ScrollView`; colección que puede crecer → `FlatList`; colección con encabezados de grupo → `SectionList`.

Imágenes

Para renderizar imágenes en Expo se utiliza **`expo-image`**. Lo mínimo son tres props — `source` (de dónde sale), `style` (qué lugar ocupa) y `contentFit` (cómo se acomoda adentro de ese cuadro).

```
import { Image } from 'expo-image';

<Image source="https://ejemplo.com/foto.jpg"
       style={{ width: '100%', height: 200 }}
       contentFit="cover" transition={200} />
```



`cover` · llena y recorta



`contain` · entra entera, deja aire

Además

Caché en disco y memoria sin configurar, y soporte de **WebP, AVIF y SVG**.

> **Toda imagen necesita dimensiones.** A diferencia de la web, React Native no reserva espacio automáticamente mientras carga: sin ancho y alto, la imagen no aparece.

Image: las props que vas a usar

Prop	Valores	Para qué
`source`	URL, `require(…)`, `{'{'} uri {'}'}`	de dónde sale la imagen
`contentFit`	**cover** · contain · fill · none · scale-down	cómo se acomoda en el cuadro
`contentPosition`	**center** · `'top right'` · `{'{'} top: 8 {'}'}`	desde qué punto recorta
`transition`	número en ms, o `{'{'} duration, effect {'}'}`	fundido al aparecer o cambiar
`placeholder`	blurhash o thumbhash	qué se ve mientras carga
`cachePolicy`	none · disk · memory · memory-disk	dónde se guarda
`priority`	low · **normal** · high	orden de descarga
`recyclingKey`	texto	limpia al reciclar filas de una lista
`tintColor`	un color	tiñe la figura — para íconos
`blurRadius`	número (**0** = sin desenfoque)	desenfoca
`alt`	texto	lo que lee el lector de pantalla

> En **negrita**, el valor por defecto. `contentFit` y `contentPosition` son `object-fit` y `object-position` de CSS con otro nombre — ya los conocés.

Un formulario sin `<form>`

No hay `<form>`, ni evento `submit`, ni `name`, ni `FormData`. Hay campos controlados sueltos y un estado que los junta; el botón, cuando lo tocan, llama a tu función con lo que haya ahí.

```
const [email, setEmail] = useState('');
const [clave, setClave] = useState('');

<TextInput
  value={email}
  onChangeText={setEmail}
  placeholder="tu@email.com"
  keyboardType="email-address"
  autoCapitalize="none"
  autoComplete="email"
/>
<TextInput value={clave} onChangeText={setClave} secureTextEntry />
<Pressable onPress={() => ingresar(email, clave)}>
  <Text>Ingresar</Text>
</Pressable>
```

> Un `useState` por campo funciona bien hasta tres o cuatro. Más que eso, o con reglas de validación, el código empieza a repetirse — y eso es exactamente lo que resuelve la slide que sigue.

Formularios · a mano

Dos campos con validación, sin librerías: un estado por campo, un objeto de errores y una función que valida antes de enviar.

```
import { useState } from 'react';
import { TextInput, Text, Pressable } from 'react-native';

const [email, setEmail] = useState('');
const [clave, setClave] = useState('');
const [errores, setErrores] = useState({});

const enviar = () => {
  const e = {};
  if (!email.includes('@')) e.email = 'Email inválido';
  if (clave.length < 8)     e.clave = 'Mínimo 8 caracteres';
  setErrores(e);
  if (Object.keys(e).length > 0) return;
  // enviar...
};

<TextInput value={email} onChangeText={setEmail} />
{errores.email && <Text>{errores.email}</Text>}
<TextInput value={clave} onChangeText={setClave} secureTextEntry />
{errores.clave && <Text>{errores.clave}</Text>}
<Pressable onPress={enviar}><Text>Ingresar</Text></Pressable>
```

> Funciona, y para dos campos está bien. Lo que no escala es la validación: cada regla nueva es una línea más en `enviar` y una entrada más en `errores`.

Formularios · `react-hook-form` + `zod`

El mismo formulario, pero las reglas viven en un **esquema** y la librería se encarga del resto: validar, guardar los valores y decir qué falló.

```
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import * as z from 'zod';

const esquema = z.object({
  email: z.email({ error: 'Email inválido' }),
  clave: z.string().min(8, { error: 'Mínimo 8 caracteres' }),
});
const { control, handleSubmit, formState: { errors } } =
  useForm({ resolver: zodResolver(esquema) });

<Controller name="email" control={control}
  render={({ field: { value, onChange } }) => (
    <TextInput value={value} onChangeText={onChange} />)} />
{errors.email && <Text>{errors.email.message}</Text>}
<Controller name="clave" control={control}
  render={({ field: { value, onChange } }) => (
    <TextInput value={value} onChangeText={onChange} secureTextEntry />)} />
{errors.clave && <Text>{errors.clave.message}</Text>}

<Pressable onPress={handleSubmit(ingresar)}><Text>Ingresar</Text></Pressable>
```

> Con dos campos empatan. Con ocho, cada uno con sus reglas, el de la slide anterior se multiplica y este es **el mismo código**: lo único que crece es el esquema. Y ese esquema además **tipa** el formulario — si escribís `errors.emial`, TypeScript te avisa.

Manejo del teclado

El problema

El teclado aparece y tapa el campo donde se está escribiendo. En iOS y Android se comporta distinto.

La solución

`KeyboardAvoidingView` para casos simples; `react-native-keyboard-controller` cuando se necesita control fino o animaciones que sigan al teclado.

```
<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={{ flex: 1 }}
>
  <TextInput ... />
</KeyboardAvoidingView>
```

Expo UI ESTABLE DESDE SDK 56

`@expo/ui` es un conjunto de componentes que se dibujan **directamente con SwiftUI en iOS y con Jetpack Compose en Android**. Los componentes de React Native están inspirados en los del sistema; estos **son** los del sistema, no una imitación hecha en JavaScript.

Hasta SDK 55

Un selector de fecha, un slider o una hoja inferior había que armarlos a mano o traer una dependencia de la comunidad: cada una con su propio calendario de versiones y su propia forma de romperse al subir de SDK.

Desde SDK 56

Vienen en el SDK y son estables. Y `@expo/ui/community` trae reemplazos **con la misma API** de las librerías más usadas: cambiás el import y sacás la dependencia del proyecto.

Acá sí importa sobre qué sistema corre la app. Los componentes universales se importan de `@expo/ui` y se resuelven solos. Los propios de una plataforma salen de `@expo/ui/swift-ui` o de `@expo/ui/jetpack-compose`, y ahí la decisión es tuya.

Expo UI en código

Una pantalla de configuración con controles del sistema.

```
import { Host, Column, Button, Switch, Picker } from '@expo/ui';

<Host style={{ height: 220 }}>
  <Column spacing={12}>
    <Switch label="Notificaciones"
            value={activo} onValueChange={setActivo} />
    <Picker selectedValue={frecuencia} onValueChange={setFrecuencia}>
      <Picker.Item label="Diaria"  value="diaria" />
      <Picker.Item label="Semanal" value="semanal" />
    </Picker>
    <Button label="Guardar" onPress={guardar} />
  </Column>
</Host>
```

Qué gana

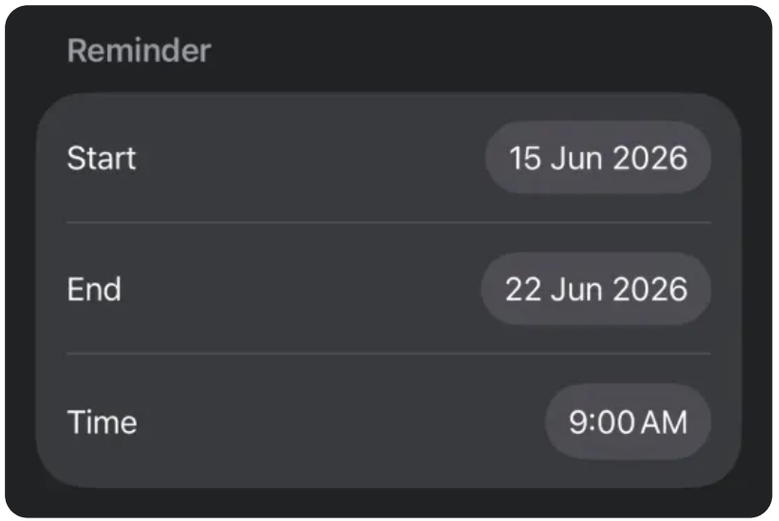
El control se ve y se comporta como el del sistema, y hereda su accesibilidad y sus animaciones sin que vos hagas nada.

Qué cuesta

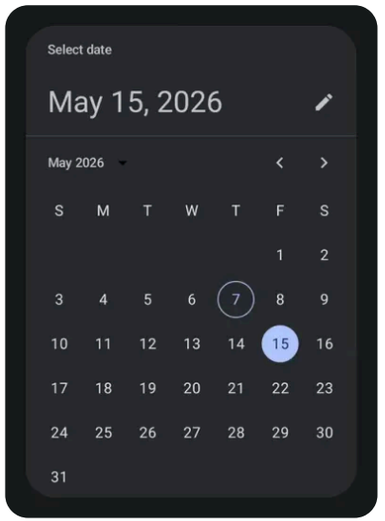
Todo va adentro de un `Host`, con alto propio: el layout de adentro no es el Flexbox de React Native, es el de SwiftUI o Compose.

El mismo control, dos sistemas

Un selector de fecha, bajando al módulo específico de cada plataforma. No es el mismo componente con otra piel: cambian el nombre, las props y hasta el tipo del valor.



iOS · `DatePicker` de SwiftUI



Android · `DateTimePicker` de Jetpack Compose

	`@expo/ui/swift-ui`	`@expo/ui/jetpack-compose`
componente	`DatePicker`	`DateTimePicker`
valor	`selection` — un `Date`	`initialDate` — texto ISO
cambio	`onDateChange`	`onDateSelected`
estilo	modificador `datePickerStyle('graphical')`	prop `variant="picker"`

UNIDAD 3 · PARTE 2

Estilos, temas y diseño responsivo

De `StyleSheet` a una aplicación que se adapta.

La base: `StyleSheet`

EN LÍNEA

```
<View style={{ padding: 16 }}>
```

Cómodo para un ajuste puntual. Crea un objeto nuevo en cada render, así que no conviene en listas largas.

STYLESHEET.CREATE

```
const e = StyleSheet.create({
  caja: { padding: 16 },
});

<View style={e.caja}>
```

Se define una vez, fuera del componente.

> Se combinan pasando un arreglo — `style={[e.caja, e.destacada]}` — y así se aplican estilos condicionales, que vemos en la próxima slide.

Lo que cambia respecto de CSS

Sin unidades

`padding: 16`, no `16px`. El número son density-independent pixels: el sistema los escala según la densidad de la pantalla.

Flexbox por defecto

Toda `View` ya es `display: flex` con `flexDirection: 'column'` — al revés que en web, donde es `row`.

Sin atajos

No existe `margin: 0 auto` ni `border: 1px solid red`. Va propiedad por propiedad: `borderWidth`, `borderColor`, `borderStyle`.

Sin cascada ni herencia

Un `color` puesto en una `View` **no llega al `Text` de adentro**. Cada componente define lo suyo.

> La falta de herencia es lo que más sorprende viniendo de web. La única excepción es un `<Text>` anidado dentro de otro `<Text>`: ahí sí hereda tipografía y color.

Estilos condicionales

Un chip que se selecciona al tocarlo. Apretalo para ver los dos estados.

Ofertas

EL COMPONENTE

```
export function Chip({ etiqueta }) {
  const [activo, setActivo] = useState(false);

  return (
    <Pressable
      onPress={() => setActivo(!activo)}
      style={[e.chip, activo && e.chipActivo]}
    >
      <Text style={[e.texto, activo && e.textoActivo]}>
        {etiqueta}
      </Text>
    </Pressable>
  );
}
```

LOS ESTILOS

```
const e = StyleSheet.create({
  chip: {
    paddingVertical: 8,
    paddingHorizontal: 18,
    borderRadius: 999,
    borderWidth: 2,
    borderColor: '#D8E0E8',
    backgroundColor: '#FFFFFF',
  },
  chipActivo: {
    borderColor: '#0B5394',
    backgroundColor: '#0B5394',
  },
  texto:       { color: '#5C6B7A' },
  textoActivo: { color: '#FFFFFF' },
});
```

El arreglo se resuelve de izquierda a derecha: **lo de la derecha pisa lo de la izquierda**. Un `false` se ignora, así que `activo && e.chipActivo` aplica el estilo sólo cuando corresponde.

El panorama de librerías

Sobre `StyleSheet` se construyeron varias librerías. Todas resuelven lo mismo: **temas, variantes y diseño responsivo**.

Unistyles

API casi idéntica a `StyleSheet`. Sin re-renders, con temas y breakpoints incluidos. Requiere development build.

NativeWind

Tailwind en React Native: clases utilitarias en `className`. La más usada del ecosistema.

Tamagui

Sistema de diseño completo, con componentes propios y compilador. Más ambicioso y más opinado.

Styled Components CÓDIGO HEREDADO

CSS-in-JS con template literals. Lo van a encontrar en proyectos existentes; no es la opción a elegir en uno nuevo.

Temas

React Native te dice en qué modo está el sistema. El resto —qué colores usar— lo definís vos.

```
import { useColorScheme } from 'react-native';

const temas = {
  claro:  { fondo: '#FFFFFF', texto: '#1A2530', acento: '#0B5394' },
  oscuro: { fondo: '#12191F', texto: '#E8EDF2', acento: '#7FB3E0' },
};

export function Pantalla() {
  const modo = useColorScheme();                    // 'light' | 'dark'
  const t = temas[modo === 'dark' ? 'oscuro' : 'claro'];

  return (
    <View style={[e.caja, { backgroundColor: t.fondo }]}>
      <Text style={{ color: t.texto }}>Hola</Text>
    </View>
  );
}
```

> Repetir esas dos líneas en cada pantalla no escala. La solución habitual es un **Context** que exponga el tema, y que cada componente lo lea con un hook propio: `const t = useTema();`

Diseño responsivo

Una app tiene que verse bien en un teléfono chico, en uno grande, en una tablet y **rotada**.

```
import { useWindowDimensions } from 'react-native';

export function Grilla({ productos }) {
  const { width } = useWindowDimensions();   // re-renderiza al rotar
  const columnas = width >= 768 ? 3 : width >= 576 ? 2 : 1;

  return (
    <FlatList
      data={productos}
      numColumns={columnas}
      key={columnas}                          // fuerza el remontaje al cambiar
      renderItem={({ item }) => <Tarjeta producto={item} />}
    />
  );
}
```

`useWindowDimensions` **sí re-renderiza** cuando cambia el tamaño o la orientación. Es lo que querés — pero si lo usás en un componente que está dentro de una lista larga, ese costo se multiplica.

Unistyles opcional

Resuelve temas, breakpoints y variantes **sin escribir el andamiaje**. Es `StyleSheet` con capas encima: se adopta cambiando el import.

TEMAS Y BREAKPOINTS

```
- import { StyleSheet } from 'react-native'
+ import { StyleSheet } from 'react-native-unistyles'
```

```
const e = StyleSheet.create(theme => ({
  grilla: {
    backgroundColor: theme.colors.fondo,
    flexDirection: { xs: 'column', md: 'row' },
    padding: { xs: 12, md: 24 },
  },
}));
```

Sin Context, sin `useWindowDimensions`, sin re-renders.

VARIANTES

```
chip: {
  borderRadius: 999,
  variants: {
    activo: {
      true:  { backgroundColor: theme.colors.acento },
      false: { backgroundColor: theme.colors.fondo },
    },
  },
},

e.useVariants({ activo });
```

El estado se declara, en vez de armar arreglos condicionales.

Unistyles: qué cuesta adoptarlo

No corre en Expo Go

Tiene código nativo propio. Hay que instalar dos dependencias, agregar el plugin de Babel y correr `npx expo prebuild --clean`, y a partir de ahí trabajar con un **development build**.

Lo que ganás

Temas y breakpoints sin andamiaje, variantes declarativas, y cambios de estilo que no re-renderizan ningún componente.

Es opcional. Con `StyleSheet`, un Context para el tema y `useWindowDimensions` se llega al mismo resultado. Si el proyecto es chico o prefieren no salir de Expo Go, es una decisión perfectamente válida.

NativeWind opcional

Tailwind en React Native. Las clases se escriben en `className` y un compilador de Babel las convierte en objetos de `StyleSheet` en tiempo de build.

EL MISMO CHIP

```
<Pressable
  className={`px-5 py-2 rounded-full border-2
    ${activo
      ? 'bg-blue-800 border-blue-800'
      : 'bg-white border-slate-300'}`}
>
  <Text className={activo ? 'text-white' : 'text-slate-500'}>
    {etiqueta}
  </Text>
</Pressable>
```

CUÁNDO CONVIENE

- Ya trabajan con Tailwind en web.
- Se comparte el sistema de diseño entre web y móvil: mismo `tailwind.config`.
- **Sí funciona en Expo Go**: es un plugin de Babel, no tiene código nativo.

En contra: el JSX se llena de clases y las condicionales quedan como cadenas de texto largas.

iOS 26: Liquid Glass

Apple rediseñó el sistema alrededor de un material translúcido que refracta lo que tiene detrás. Expo lo expone con `expo-glass-effect`.

```
import { GlassView, isLiquidGlassAvailable } from 'expo-glass-effect';

{isLiquidGlassAvailable()
  ? <GlassView style={e.barra} glassEffectStyle="regular" />
  : <View style={[e.barra, { backgroundColor: '#F2F2F7' }]} />}
```

> Es **sólo iOS 26 o superior**: abajo de esa versión `GlassView` se comporta como una `View` común, así que el diseño tiene que funcionar igual sin el efecto. En Android no existe.

UNIDAD 3 · PARTE 3

Gestos y animaciones

Lo que separa una app que funciona de una que se siente bien.

Gestos

`react-native-gesture-handler` reconoce toques, desplazamientos, pellizcos y rotaciones — y lo hace **en el hilo nativo**, sin esperar a JavaScript.

```
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

const arrastrar = Gesture.Pan()
  .onUpdate(e => {
    x.value = e.translationX;
    y.value = e.translationY;
  });

<GestureDetector gesture={arrastrar}>
  <Animated.View style={estiloAnimado} />
</GestureDetector>
```

> Los gestos se **componen**: `Gesture.Simultaneous()`, `Gesture.Race()`, `Gesture.Exclusive()`. Así se arma, por ejemplo, pellizcar para hacer zoom mientras se arrastra.

Animaciones con Reanimated 4

```
import Animated, { useSharedValue, useAnimatedStyle,
                   withSpring } from 'react-native-reanimated';

const escala = useSharedValue(1);

const estilo = useAnimatedStyle(() => ({
  transform: [{ scale: escala.value }],
}));

<Pressable onPress={() => { escala.value = withSpring(1.2); }}>
  <Animated.View style={estilo} />
</Pressable>
```

> Un **shared value** vive fuera del estado de React: cambiarlo no dispara un render. La animación corre en el hilo de UI, así que se mantiene fluida aunque JavaScript esté ocupado.

UNIDAD 3 · PARTE 4

Accesibilidad

Una pantalla que sólo funciona mirándola está a medio terminar.

Qué es y por qué acá

Quién la usa

Los lectores de pantalla del sistema — **VoiceOver** en iOS, **TalkBack** en Android — leen la pantalla en voz alta y se navegan por gestos.

También el que agrandó la tipografía del sistema, el que invierte los colores y el que apagó las animaciones.

Por qué en esta unidad

Porque se resuelve **mientras se arma la pantalla**, con las mismas props del componente. Agregada al final es un rediseño.

Y porque lo mismo que la hace accesible la hace **testeable**: las consultas de React Native Testing Library buscan por rol y por etiqueta.

> El sistema operativo ya hace casi todo. Lo que hace tu app es **describirle qué es cada cosa**: eso es la accesibilidad en React Native.

Las props que resuelven el 90%

```
<Pressable
  accessible                              // es UN elemento, no tres
  role="button"                           // qué es
  accessibilityLabel="Agregar al carrito" // cómo se llama
  accessibilityHint="Suma el producto y vuelve al listado"
  onPress={agregar}>
  <Ionicons name="cart" size={24} />
</Pressable>
```

Se lee así

«Agregar al carrito, botón» — y después, si el usuario tiene las pistas activadas, la hint.

`role`, no `accessibilityRole`

Los dos existen y hacen lo mismo. `role` es la forma nueva, tiene precedencia sobre la otra y usa los mismos nombres de ARIA que ya vienen de web.

Estado, valor y cambios

Un control que cambia tiene que **contar** que cambió.

```
<Pressable role="switch"
  accessibilityLabel="Notificaciones"
  accessibilityState={{ checked: activo, disabled: !editable }} />

<Slider role="adjustable"
  accessibilityLabel="Volumen"
  accessibilityValue={{ min: 0, max: 100, now: volumen }} />

{/* el resultado de una búsqueda, que aparece sin que el foco se mueva */}
<Text accessibilityLiveRegion="polite">{n} resultados</Text>
```

> Los alias de ARIA también funcionan: `aria-label`, `aria-checked`, `aria-disabled`, `aria-hidden`, `aria-live`. Si venís de web, escribí como escribías.

Los cuatro errores de siempre

Botón que es sólo un ícono

Sin `accessibilityLabel` el lector anuncia «botón» y nada más. Es el más frecuente de todos.

Imagen decorativa que se lee

Si no aporta información va con `aria-hidden`. Si aporta, va con etiqueta.

Área de toque diminuta

El mínimo es **44 pt** en iOS y **48 dp** en Android. Se agranda con `hitSlop`, sin tocar el diseño.

Contraste al límite

El gris claro sobre blanco que se ve bien en el monitor desaparece al sol. El piso es **4,5:1** para texto normal.

> Un quinto, más sutil: envolver media pantalla en un `View` con `accessible`. El lector lo trata como **un** elemento y lee todo de corrido, sin poder entrar a los botones de adentro.

Cómo se prueba

Android · TalkBack

Ajustes → Accesibilidad → TalkBack. Se prende y apaga manteniendo **las dos teclas de volumen 3 segundos**.

El emulador no lo trae: hace falta una imagen con Play Store e instalarlo, o usar un teléfono real.

iOS · VoiceOver

Ajustes → Accesibilidad → VoiceOver. Conviene dejarlo en el atajo de accesibilidad: **triple clic** al botón lateral.

No corre en el simulador. Desde Xcode se usa el Accessibility Inspector, que no es lo mismo pero sirve.

Prueba de dos minutos, sin herramientas: prendé el lector, **apagá la pantalla** y tratá de completar la tarea principal de tu app. Lo que no puedas hacer, no lo puede hacer nadie que dependa del lector.

UNIDAD 3

Lo que sigue

- **Navegación:** deck complementario de esta misma unidad.
- Práctica: pantallas con tema claro/oscuro y diseño adaptable.
- **Unidad 4:** cámara, ubicación, sensores y sistema de archivos.

Documentación: `docs.expo.dev/router` · `docs.expo.dev/versions/latest/sdk/ui` · `reactnative.dev/docs/accessibility` · `unistyl.es`